

Unit VIII: Security Auditing

Contents

- 8.1 Security Audit
- 8.2 Security Auditing Architecture
- 8.3 Security Audit Trail
- 8.4 Implementing Logging Function
- 8.5 Audit Trail Analysis

8.1 Security Audit

Security Audit is an independent review and examination of a system's records and activities to determine the adequacy of system controls, ensure compliance with established security policy and procedures, detect breaches in security services, and recommend any changes that are indicated for countermeasures.

The basic audit objective is to establish accountability for system entities that initiate or participate in security-relevant events and actions. Thus, means are needed to generate and record a security audit trail and to review and analyze the audit trail to discover and investigate attacks and security compromises.

Security auditing is a form of auditing that focuses on the security of an organization's information technology (IT) assets. This function is a key element in computer security.

Security auditing can:

- Provide a level of assurance concerning the proper operation of the computer with respect to security.
- Generate data that can be used in after-the-fact analysis of an attack, whether successful or unsuccessful.
- Provide a means of assessing inadequacies in the security service.
- Provide data that can be used to define anomalous behavior.
- Maintain a record useful in computer forensics.

8.2 Security Auditing Architecture

Security auditing architecture includes the elements that required for auditing. It shows security auditing in broader context. ITU-T Recommendation X.816 develops a model that shows the elements of the security auditing function and their relationship to security alarms. Figure 18.1 depicts the model. The key elements are as follows:

- **Event discriminator:** This is logic embedded into the software of the system that monitors system activity and detects security-related events that it has been configured to detect.
- **Audit recorder:** For each detected event, the event discriminator transmits the information to an audit recorder. The model depicts this transmission as being in the form of a message. The audit could also be done by recording the event in a shared memory area.
- **Alarm processor:** Some of the events detected by the event discriminator are defined to be alarm events. For such events, an alarm is issued to an alarm processor. The alarm processor takes some action based on the alarm. This action is itself an auditable event, and so is transmitted to the audit recorder.

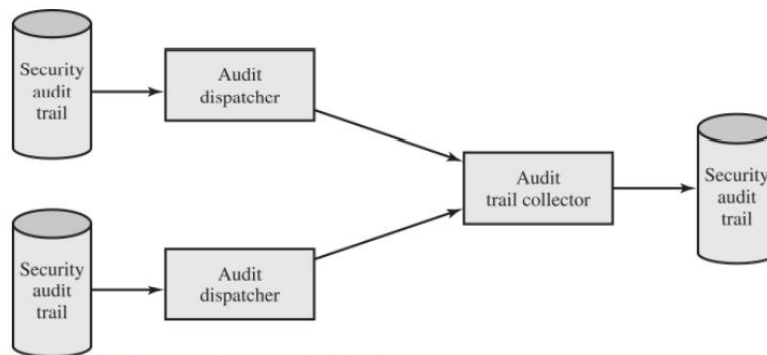


Figure 18.2 Distributed Audit Trail Model (X.816)

Security Auditing Functions

It is useful to look at another breakdown of the security auditing function, developed as part of the Common Criteria specification [CCPS12a]. Figure 18.3 shows a breakdown of security auditing into six major areas, each of which has one or more specific functions:

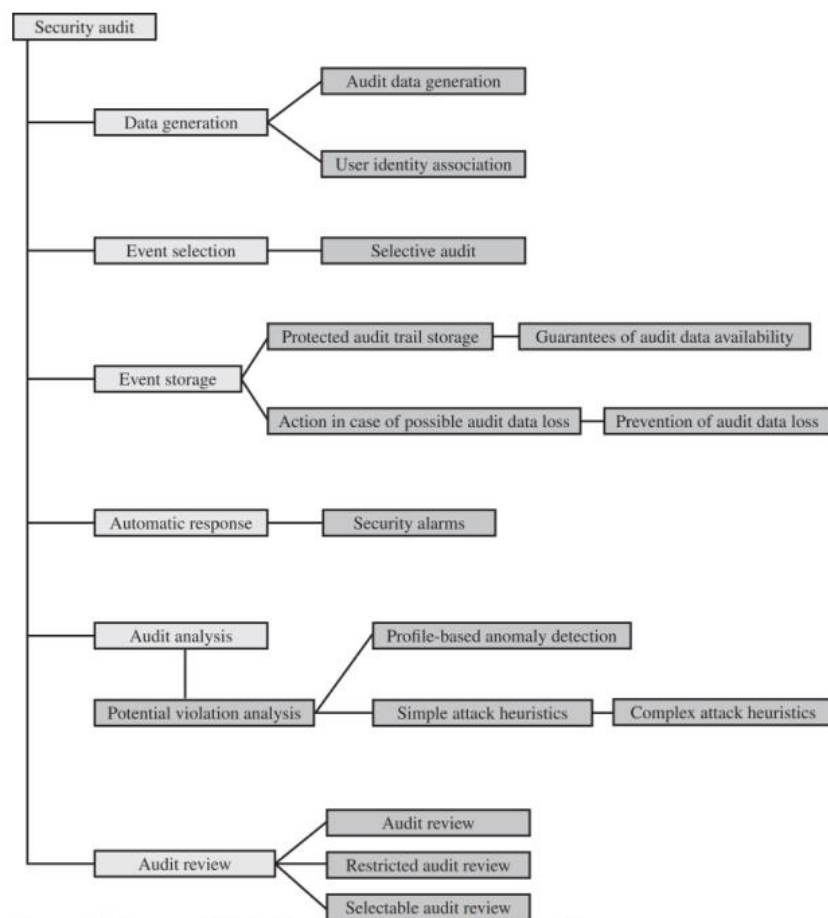


Figure 18.3 Common Criteria Security Audit Class Decomposition

- **Data generation:** Identifies the level of auditing, enumerates the types of auditable events, and identifies the minimum set of audit-related information provided. This function must also deal with the conflict between security and privacy and specify for which events the identity of the user associated with an action is included in the data generated as a result of an event.
- **Event selection:** Inclusion or exclusion of events from the auditable set. This allows the system to be configured at different levels of granularity to avoid the creation of an unwieldy audit trail.

- **Event storage:** Creation and maintenance of the secure audit trail. The storage function includes measures to provide availability and to prevent loss of data from the audit trail.
- **Automatic response:** Defines reactions taken following detection of events that are indicative of a potential security violation.
- **Audit analysis:** Provided via automated mechanisms to analyze system activity and audit data in search of security violations. This component identifies the set of auditable events whose occurrence or accumulated occurrence indicates a potential security violation. For such events, an analysis is done to determine if a security violation has occurred; this analysis uses anomaly detection and attack heuristics.
- **Audit review:** As available to authorized users to assist in audit data review. The audit review component may include a selectable review function that provides the ability to perform searches based on a single criterion or multiple criteria with logical (i.e., and/or) relations, sort audit data, and filter audit data before audit data are reviewed. Audit review may be restricted to authorized users.

8.3 Security Audit Trail

Security Audit Trail is a chronological record of system activities that is sufficient to enable the reconstruction and examination of the sequence of environments and activities surrounding or leading to an operation, procedure, or event in a security relevant transaction from inception to final results.

Audit trails maintain a record of system activity.

What to Collect?

The choice of data to collect is determined by a number of requirements. One issue is the amount of data to collect, which is determined by the range of areas of interest and by the granularity of data collection. There is a trade-off here between quantity and efficiency. The more data are collected, the greater is the performance penalty on the system. Larger amounts of data may also unnecessarily burden the various algorithms used to examine and analyze the data. Further, the presence of large amounts of data creates a temptation to generate security reports excessive in number or length.

With these cautions in mind, the first order of business in security audit trail design is the selection of data items to capture. These may include:

- Events related to the use of the auditing software.
- Events related to the security mechanisms on the system.
- Any events that are collected for use by the various security detection and prevention mechanisms. These include items relevant to intrusion detection and items related to firewall operation.
- Events related to system management and operation. Operating system access

The auditable items suggested in X.816 are:

1. Security-related events related to a specific connection
 - Connection requests
 - Connection confirmed
 - Disconnection requests
 - Disconnection confirmed
 - Statistics appertaining to the connection

2. Security-related events related to the use of security services
 - Security service requests
 - Security mechanisms usage
 - Security alarms
3. Security-related events related to management
 - Management operations
 - Management notifications
4. The list of audible events should include at least
 - Deny access
 - Authenticate
 - Change attribute
 - Create object
 - Delete object
 - Modify object
 - Use privilege
5. In terms of the individual security services, the following security-related events are important
 - Authentication: verify success
 - Authentication: verify fail
 - Access control: decide access success
 - Access control: decide access fail
 - Nonrepudiation: nonrepudiable origination of message
 - Nonrepudiation: nonrepudiable receipt of message
 - Nonrepudiation: unsuccessful repudiation of event
 - Nonrepudiation: successful repudiation of event
 - Integrity: use of shield
 - Integrity: use of unshield
 - Integrity: validate success
 - Integrity: validate fail
 - Confidentiality: use of hide
 - Confidentiality: use of reveal
 - Audit: select event for auditing
 - Audit: deselect event for auditing
 - Audit: change audit event selection criteria

Monitoring areas suggested in ISO 27002 are:

1. user IDs
2. system activities
3. dates, times, and details of key events, for example, log-on and log-off
4. device identity or location if possible and system identifier
5. records of successful and rejected system access attempts
6. records of successful and rejected data and other resource access attempts
7. changes to system configuration
8. use of privileges
9. use of system utilities and applications
10. files accessed and the kind of access
11. network addressees and protocols

12. alarms raised by the access control system
13. activation and de-activation of protection systems, such as anti-virus systems and intrusion detection systems
14. records of transactions executed by users in applications

Categories of audit trails

- System-level audit trails
- Application-level audit trails
- User-level audit trails
- Physical access audit trails

System-Level Audit Trails

System-level audit trails are generally used to monitor and optimize system performance but can serve a security audit function as well. The system enforces certain aspects of security policy, such as access to the system itself. A system-level audit trail should capture data such as login attempts, both successful and unsuccessful, devices used, and OS functions performed. Other system-level functions may be of interest for auditing, such as system operation and network performance indicators.

Application-Level Audit Trails

Application-level audit trails may be used to detect security violations within an application or to detect flaws in the application's interaction with the system. For critical applications, or those that deal with sensitive data, an application-level audit trail can provide the desired level of detail to assess security threats and impacts. For example, for an e-mail application, an audit trail can record sender and receiver, message size, and types of attachments. An audit trail for a database interaction using SQL (Structured Query Language) queries can record the user, type of transaction, and even individual tables, rows, columns, or data items accessed.

User-Level Audit Trails

A user-level audit trail traces the activity of individual users over time. It can be used to hold a user accountable for his or her actions. Such audit trails are also useful as input to an analysis program that attempts to define normal versus anomalous behavior.

A user-level audit trail can record user interactions with the system, such as commands issued, identification and authentication attempts, and files and resources accessed. The audit trail can also capture the user's use of applications.

Physical Access Audit Trails

Audit trails can be generated by equipment that controls physical access and then transmits them to a central host for subsequent storage and analysis. Examples are card-key systems and alarm systems. NIST SP 800-12 lists the following as examples of the type of data of interest:

- The date and time the access was attempted or made should be logged, as should the gate or door through which the access was attempted or made, and the individual (or user ID) making the attempt to access the gate or door.
- Invalid attempts should be monitored and logged by noncomputer audit trails just as they are for computer system audit trails. Management should be made aware if someone attempts to gain access during unauthorized hours.

- Logged information should also include attempts to add, modify, or delete physical access privileges (e.g., granting a new employee access to the building or granting transferred employees access to their new office [and, of course, deleting their old access, as applicable]).
- As with system and application audit trails, auditing of noncomputer functions can be implemented to send messages to security personnel indicating valid or invalid attempts to gain access to controlled spaces. In order not to desensitize a guard or monitor, all access should not result in messages being sent to a screen. Only exceptions, such as failed access attempts, should be highlighted to those monitoring access.

8.4 Implementing Logging Function

The foundation of a security auditing facility is the initial capture of the audit data. This requires that the software include hooks, or capture points, that trigger the collection and storage of data as preselected events occur. Such an audit collection or logging function is dependent on the nature of the software and will vary depending on the underlying operating system and the applications involved.

Logging at the System Level

Much of the logging at the system level can be implemented using existing facilities that are part of the operating system.

Windows Event Log

An event in Windows Event Log is an entity that describes some interesting occurrence in a computer system. Events contain a numeric identification code, a set of attributes (task, opcode, level, version, and keywords), and optional user-supplied data. Windows is equipped with three types of event logs:

- **System event log:** Used by applications running under system service accounts (installed system services), drivers, or a component or application that has events that relate to the health of the computer system.
- **Application event log:** Events for all user-level applications. This log is not secured and it is open to any applications. Applications that log extensive information should define an application-specific log.
- **Security event log:** The Windows Audit Log. This event log is for exclusive use of the Windows Local Security Authority. User events may appear as audits if supported by the underlying application.

Windows allows the system user to enable auditing in nine different categories:

1. **Account logon events:** User authentication activity from the perspective of the system that validated the attempt. Examples: authentication granted; authentication ticket request failed; account mapped for logon; account could not be mapped for logon. Individual actions in this category are not particularly instructive, but large numbers of failures may indicate scanning activity, brute-force attacks on individual accounts, or the propagation of automated exploits.
2. **Account management:** Administrative activity related to the creation, management, and deletion of individual accounts and user groups. Examples: user account created; change password attempt; user account deleted; security enabled global group member added; domain policy changed.

3. **Directory service access:** User-level access to any Active Directory object that has a System Access Control List defined. An SACL creates a set of users and user groups for which granular auditing is required.
4. **Logon events:** User authentication activity, either to a local machine or over a network, from the system that originated the activity. Examples: successful user logon; logon failure, unknown username, or bad password; logon failure, because account is disabled; logon failure, because account has expired; logon failure, user not allowed to logon at this computer; user logoff; logon failure, account locked out.
5. **Object access:** User-level access to file system and registry objects that have System Access Control Lists defined. Provides a relatively easy way to track read access, as well as changes, to sensitive files, integrated with the operating system. Examples: object open; object deleted.
6. **Policy changes:** Administrative changes to the access policies, audit configuration, and other system-level settings. Examples: user right assigned; new trusted domain; audit policy changed.
7. **Privilege use:** Windows incorporates the concept of a user right, granular permission to perform a particular task. If you enable privilege use auditing, you record all instances of users exercising their access to particular system functions (creating objects, debugging executable code, or backing up the system). Examples: specified privileges were added to a user's access token (during logon); a user attempted to perform a privileged system service operation.
8. **Process tracking:** Generates detailed audit information when processes start and finish, programs are activated, or objects are accessed indirectly. Examples: new process was created; process exited; auditable data was protected; auditable data was unprotected; user attempted to install a service.
9. **System events:** Records information on events that affect the availability and integrity of the system, including boot messages and the system shutdown message. Examples: system is starting; Windows is shutting down; resource exhaustion in the logging subsystem; some audits lost; audit log cleared.

Syslog

Syslog is UNIX's general-purpose logging mechanism found on all UNIX variants and Linux. It consists of the following elements:

- **syslog():** An application program interface (API) referenced by several standard system utilities and available to application programs
- **logger:** A UNIX command used to add single-line entries to the system log
- **/etc/syslog.conf:** The configuration file used to control the logging and routing of system log events
- **syslogd:** The system daemon used to receive and route system log events from syslog() calls and logger commands.

Different UNIX implementations will have different variants of the syslog facility, and there are no uniform system log formats across systems.

Logging at the Application Level

Applications, especially those with a certain level of privilege, present security problems that may not be captured by system-level or user-level auditing data. Application-level vulnerabilities constitute a large percentage of reported vulnerabilities on security mailing lists. One type of vulnerability that can be

exploited is the all-too-frequent lack of dynamic checks on input data, which make possible buffer overflow.

Auditing at the system level does not provide the level of detail to catch application logic error behavior. Further, intrusion detection systems look for attack signatures or anomalous behavior that would fail to appear with attacks based on application logic errors. For both detection and auditing purposes, it may be necessary to capture in detail the behavior of an application, beyond its access to system services and file systems. The information needed to detect application-level attacks may be missing or too difficult to extract from the low-level information included in system call traces and in the audit records produced by the operating system.

There are two approaches to collecting audit data from applications: interposable libraries, and dynamic binary rewriting.

Interposable Libraries

It provides for application-level auditing by creating new procedures that intercept calls to shared library functions in order to instrument the activity. Interposition allows the generation of audit data without needing to recompile either the system libraries or the application of interest. Thus, audit data can be generated without changing the system's shared libraries or needing access to the source code for the executable on which the interposition is to be performed. This approach can be used on any UNIX or Linux variant and on some other operating systems.

Dynamic Binary Rewriting

The interposition technique is designed to work with dynamically linked shared libraries. It cannot intercept function calls of statically linked programs unless all programs in the system are relinked at the time that the audit library is introduced. Dynamic binary rewriting can be used with both statically and dynamically linked programs.

Dynamic binary rewriting is a postcompilation technique that directly changes the binary code of executables. The change is made at load time and modifies only the memory image of a program, not the binary program file on secondary storage. As with the interposition technique, dynamic binary rewriting does not require recompilation of the application binary. Audit module selection is postponed until the application is invoked, allowing for flexible selection of the auditing configuration.

8.5 Audit Trail Analysis

Programs and procedures for audit trail analysis vary widely, depending on the system configuration, the areas of most concern, the software available, the security policy of the organization, and the behavior patterns of legitimate users and intruders.

1. Preparation

To perform useful audit analysis, the analyst or security administrator needs an understanding of the information available and how it can be used. NIST SP 800-92 offers some useful advice in this regard:

Understanding Log entries

The security administrator (or other individual reviewing and analyzing logs) needs to understand the context surrounding individual log entries. Relevant information may reside in other entries in the same log, entries in other logs, and nonlog sources such as configuration management entries. The

administrator should understand the potential for unreliable entries, such as from a security package that is known to generate frequent false positives when looking for malicious activity.

Understanding the context

To perform effective reviews and analysis, administrators should have solid understanding of each of the following from training or hands-on experience:

- The organization's policies regarding acceptable use, so administrators can recognize violations of the policies.
- The security software used by their hosts, including the types of security-related events that each program can detect and the general detection profile of each program (e.g., known false positives).
- The operating systems and major applications (e.g., e-mail, Web) used by their hosts, particularly each OS's and major application's security and logging capabilities and characteristics.
- The characteristics of common attack techniques, especially how the use of these techniques might be recorded on each system.
- The software needed to perform analysis, such as log viewers, log reduction scripts, and database query tools.

2. Timing

Audit trails can be used in multiple ways. The type of analysis depends, at least in part, on when the analysis is to be done. The possibilities include the following:

- **Audit trail review after an event:** This type of review is triggered by an observed event, such as a known system or application software problem, a known violation of existing security policy by a user, or some unexplained system or user problem. The review can gather information to elaborate on what is known about the event, to diagnose the cause or the problem, and to suggest remedial action and future countermeasures. This type of review focuses on the audit trail entries that are relevant to the specific event.
- **Periodic review of audit trail data:** This type of review looks at all of the audit trail data or at defined subsets of the data, and has many possible objectives. Examples of objectives include looking for events or patterns that suggest a security problem, developing a profile of normal behavior and searching for anomalous behavior, and developing profiles by individual user to maintain a permanent record by user.
- **Real-time audit analysis:** Audit analysis tools can also be used in a real-time or near-real-time fashion. Real-time analysis is part of the intrusion detection function

3. Audit Review

Distinct from an analysis of audit trail data using data reduction and analysis tools is the concept of audit review. An audit review capability enables an administrator to read information from selected audit records. The Common Criteria specification [CCPS12a] calls for a capability that allows prestorage or poststorage audit selection and includes the ability to selectively review the following:

- The actions of one or more users (e.g., identification, authentication, system entry, and access control actions)
- The actions performed on a specific object or system resource
- All or a specified set of audited exceptions
- Actions associated with a specific system or security attribute

Audit review can be focused on records that match certain attributes, such as user or user group, time window, type of record, and so forth.

4. Approaches to Data Analysis

The spectrum of approaches and algorithms used for audit data analysis is far too broad. However, some of the major approaches are:

- **Basic Alerting:** The simplest form of an analysis is for the software to give an indication that a particular interesting event has occurred. If the indication is given in real time, it can serve as part of an intrusion detection system. For events that may not rise to the level of triggering an intrusion alert, an after-the-fact indication of suspicious activity can lead to further analysis.
- **Baselining:** Baselining is the process of defining normal versus unusual events and patterns. The process involves measuring a set of known data to compute a range of normal values. These baseline values can then be compared to new data to detect unusual shifts. Examples of activity to baseline include the following:
 - a) Amount of network traffic per protocol: total HTTP, e-mail, FTP, and so on
 - b) Logins/logouts
 - c) Accesses of admin accounts
 - d) Dynamic Host Configuration Protocol (DHCP) address management, DNS requests
 - e) Total amount of log data per hour/day
 - f) Number of processes running at any time

For example, a large increase in FTP traffic could indicate that your FTP server has been compromised and is being used maliciously by an outsider.

- **Correlation:** Another type of analysis is correlation, which seeks for relationships among events. A simple instance of correlation is, given the presence of one particular log message, to alert on the presence of a second particular message. For instance, if Snort reports a buffer overflow attempt from a remote host, a reasonable attempt at correlation would grab any messages that contain the remote host's IP address. Or the administrator might want to note any switch user (su) on an account that was logged into from a never-seen-before remote host.

Practice Questions

1. What is security audit? Explain with its importance.
2. Explain the difference between a security audit message and a security alarm.
3. Explain the security auditing architecture in detail.
4. List and briefly describe the principal security auditing functions.
5. In what area (categories of data should audit data be collected?
6. What is audit trail? Explain the different categories of audit trails.
7. What is event logging? Explain about Windows eventlog in short.
8. Explain the differences between audit review and audit analysis.
9. Explain the concept of audit trail analysis in detail.